

数据在内存中的存储方式

复习

1. 进制概念
2. 进制间相互转换

目录

1. 前言
2. 原码
3. 反码
4. 补码
5. 为什么采用补码

前言

一个数在计算机中的表示形式是二进制，这个数其实就叫机器数。机器数是带符号的，在计算机用一个数的最高位存放符号，正数为0，负数为1。比如，十进制中的数 +7，计算机字长为8位，转换成二进制就是00000111。如果是 -7，就是 10000111。一个存储的二进制码分原码、反码、补码

原码、反码和补码

- 计算机在任何情况下都只能识别二进制
- 计算机在底层存储数据的时候，一律存储的是“二进制的补码形式”

为什么采用补码形式存储数据？

补码形式效率最高。

原码

原码（0表示正数，1表示负数）

$x=1100110$, 则 $[X]_{\text{原}}=01100110$

$x=-1100111$, 则 $[X]_{\text{原}}=11100111$

反码

反码(正数的反码就是自身，负数的反码除符号位外，其他各位求反)

$x=1100110$, 则 $[x]_{\text{反}}=01100110$,

$x=-1100111$, 则 $[x]_{\text{反}}=10011000$,

反码肯定属于有符号位，相当于上面有符号位求反

补码

补码(正数的补码还是自身，负数的补码，符号位不变，其余取反，然后最低为加1)

$x=1100110$, 则 $[x]_{\text{补}}=01100110$,

$x=-1100111$, 则 $[x]_{\text{补}}=10011001$

why?

为什么计算机要采用补码的方式来存储数据呢？采用原码不是更容易理解吗？

原码来处理数据

如果使用原码来处理数据，在运算时会是怎样呢？

十进制运算： $1 - 1 = 0$

$$1 + (-1) = 0$$

二进制原码： $0000\ 0001 + 1000\ 0001 = 1000\ 0002 = -2$

很明显，对于 $1-1$ 这个十进制计算的结果并不正确，于是我们想到用反码来解决这个问题

反码处理数据

使用反码来计算：

十进制运算： $1 - 1 = 0$

$$1 + (-1) = 0$$

二进制反码：反码[0000 0001] + 反码[1111 1110] = 反码[1111 1111] = 原码[1000 0000] = -0

对于以上结果，已经大致上符合了我们的预期，但是对于完美的答案0来说，多了一个负号总是不舒服的。于是，补码隆重登场！

补码处理数据

使用补码来计算：

十进制运算： $1 - 1 = 0$

$$1 + (-1) = 0$$

二进制反码：补码[0000 0001] + 补码[1111 1111] = 补码[0000 0000] = 原码[0000 0000] = 0

完美！到此为止，我们终于获得了完美的0。

因此，计算机使用了补码来对数据进行存储

练习

-5的原码，反码，补码分别是多少？

练习

-5的原码，反码，补码分别是多少？

原码：10000101

反码：11111010

补码：11111011

知识点总结

1. 前言
2. 原码
3. 反码
4. 补码
5. 为什么采用补码

作业

1. 利用原码，反码和补码，计算 $34 - 11 = 23$

下节课预告

运算符与表达式

作业

1. 利用原码，反码和补码，计算 $34 - 11 = 23$

$$34 - 11 = 23 = 34 + (-11)$$

34的原码反码补码都相同为 0010 0010

-11的原码为1000 1011

反码为1111 0100

补码为1111 0101

两补码相加得0001 0111 = 23